

个人作业报告

题目：**编译原理**

实验课第五次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	李文军

2026 年 6 月 17 日

【摘　要】

本文基于 Oberon-0 语言定义 [1]，设计并实现了一个面向 Oberon-0 的逆向工程工具 ROSE（Reverse Oberon Software Engineering），可根据输入的 Oberon-0 源程序自动绘制调用图与流程图。工作分为四个子实验：实验一编写了覆盖全部语法构造的正确 Oberon-0 源程序 BankSystem.obr 及 22 个变异程序，讨论了保留字与关键字的区别以及文法的二义性；实验二使用 JFlex 1.8.2 自动生成词法分析器 OberonScanner（NFA 263 状态 $\rightarrow$ DFA 105 状态），成功扫描正确程序得到 683 个 Token，对 6 种词法错误全部精确检测；实验三使用 JavaCUP 0.11b 从 oberon.cup 自动生成 LALR 语法分析器（190 状态、0 冲突），实现了参数数目检查和类型推导等语义检查，对 BankSystem.obr 生成了包含 10 个过程和 35 条调用边的调用图；实验四手工设计了 Oberon-0 的翻译模式，采用链式作用域管理嵌套过程的名称查找，编写递归下降预测分析器 OberonParser，为 8 个过程生成了完整的流程图，并对自顶向下与自底向上两种分析策略进行了八维度比较。通过本次实验，我深入理解了词法分析、语法分析、语法制导翻译和自动生成工具等编译原理核心技术，同时在面向对象设计、测试驱动开发和工程文档组织方面获得了扎实的实践锻炼。

关键词： 编译原理; Oberon-0; 逆向工程; 词法分析; 语法分析; JFlex; JavaCUP; 递归下降; ROSE

目录

1 实验一: 熟悉 Oberon-0 语言定义 4
1.1 正确的 Oberon-0 源程序 4
1.2 变异程序 5
1.3 Oberon-0 语言特点讨论 6
1.4 Oberon-0 文法二义性讨论 6
1.5 自我评分参照 7
1.6 实验心得体会 7

2 实验二 8
2.1 Oberon-0 词汇表 9
2.2 词法规则的正则定义式 11
2.3 下载词法分析程序自动生成工具 JFlex 13
2.4 生成 Oberon-0 语言的词法分析程序 14
2.5 讨论不同词法分析程序生成工具的差异 15
2.6 自我评分参照 16
2.7 实验心得体会 17

3 实验三 18
3.1 下载自动生成工具 JavaCUP 19
3.2 配置和试用 JavaCUP 19
3.3 生成 Oberon-0 语法分析和语法制导翻译程序 19
3.4 讨论不同生成工具的差异 23
3.5 自我评分参照 24
3.6 实验心得体会 24

4 实验四 26
4.1 设计 Oberon-0 语言的翻译模式 27
4.2 编写递归下降预测分析程序 28

4.3 自顶向下 vs. 自底向上分析策略比较 31
4.4 自我评分参照 31
4.5 实验心得体会 32

5 实验总结 33
5.1 自我评分参照 33
5.2 知识点与技术总结 33
5.3 实验心得体会 34

参考文献 36

实验目的

本实验旨在设计并实现一个面向 Oberon-0 语言的逆向工程工具 ROSE（Reverse Oberon Software Engineering），通过完整的词法分析、语法分析和语法制导翻译过程，深入理解和掌握编译原理中的核心技术。实验分为四个子实验：

- **实验一：熟悉 Oberon-0 语言定义**。通过编写正确的 Oberon-0 源程序及其变异程序，深入理解 Oberon-0 语言的词法、语法和语义定义，讨论保留字与关键字的区别以及文法的二义性问题。
- **实验二:利用 JFlex 自动生成词法分析程序**。编写 JFlex 输入源文件 `oberon.flex`，自动生成词法分析程序，比较三种 lex 族工具。
- **实验三：利用 JavaCUP 自动生成语法分析程序**。编写 JavaCUP 输入源文件 `oberon.cup`，实现调用图自动生成，比较 yacc 族工具。
- **实验四：手工编写递归下降预测分析程序**。设计翻译模式，编写递归下降分析程序，实现流程图自动生成，比较自顶向下与自底向上策略。

项目组织

本实验项目的 ROSE 软装置提供了预定义的调用图 API（`callgraph` 包）和流程图 API（`flowchart` 包），基于开源项目 JGraph。软装置不包含异常类的完整实现（参见 Notation.md），需学生自行完善。

目录组织

- **ex1/**：实验一结果目录。
  - `testcases/BankSystem.obr`：正确的 Oberon-0 源程序，覆盖全部语法构造。
  - `testcases/BankSystem.001--022`：22 个变异程序（14 个直接构造 + 8 个 derived 派生），首行注释标注期望错误类型和变异来源。
- **ex2/**：实验二结果目录（JFlex 词法分析）。
  - `src/oberon.flex`：JFlex 输入源文件，自动生成 `OberonScanner.java`（NFA 263 → DFA 105）。
  - `src/exceptions/`：自行完善的 14 个异常类三层继承体系（软装置标记为 [PENDING]）。
  - `bin/`、`doc/`、`jflex/`：字节码、javadoc 文档、JFlex 1.8.2 工具包。
  - `gen.bat`、`build.bat`、`run.bat`、`test.bat`：生成、构建、运行和测试脚本。
- **ex3/**：实验三结果目录（语法分析 + 调用图）。
  - `src/oberon.cup`:JavaCUP 输入源文件,经 JavaCUP 0.11b 生成 `Parser.java` + `Sym.java`（190 状态，0 冲突）。
  - `src/ScannerAdapter.java`：适配 OberonScanner Token 到 CUP Symbol 接口。
  - `src/Scope.java`：链式作用域，支持嵌套过程的正向名称查找。
  - `bin/`、`doc/`、`javacup/`：字节码、javadoc 文档、JavaCUP 0.11b 工具包。
- **ex4/**：实验四结果目录（递归下降 + 流程图）。
  - `src/OberonParser.java`：手工编写的递归下降预测分析器，含翻译模式设计和 flowchart API 集成。

- `src/OberonScanner.java`、`src/Token.java`：复用实验二的词法分析器。
- `bin/`、`doc/`：字节码、javadoc 文档。
- `build.bat`、`run.bat`、`test.bat`：构建、运行和测试脚本。

1 实验一：熟悉 Oberon-0 语言定义

实验目的

本实验（实验一）旨在深入理解 Oberon-0 语言的定义，为后续词法分析、语法分析和语法制导翻译实验奠定基础。具体目标包括：

- 阅读 Oberon-0 语言的 BNF 定义和语义描述，掌握该语言的完整语法规则。
- 编写一个词法、语法和语义均正确的 Oberon-0 源程序，覆盖所有语法构造。
- 基于变异测试思想，针对每一种可能的词法、语法和语义错误构造变异程序。
- 讨论 Oberon-0 语言的特点（保留字与关键字的区别、表达式语法的独特性）。
- 分析 Oberon-0 文法的二义性，理解其与其他高级语言在语法设计上的差异。

实验目录与项目组织

实验一的项目文件均存放于 `ex1/` 子目录下，各文件说明如下：

- `readme.txt`：自述文件，包含姓名、学号、完成日期及补充说明。
- `testcases/BankSystem.obr`：词法、语法、语义均正确的 Oberon-0 源程序，覆盖了语言的全部语法构造。
- `testcases/BankSystem.001--022`：22 个变异程序（14 个直接构造 + 8 个 derived 派生），分别包含一种词法、语法或语义错误，首行注释标明期望异常类型和变异来源。文件扩展名以数字序号区分。

1.1 正确的 Oberon-0 源程序

根据 Oberon-0 语言的 BNF 定义，编写了一个模拟银行账户管理系统的 Oberon-0 源程序 `BankSystem.obr`，覆盖了 Oberon-0 语言的所有主要语法构造：

- **模块声明**：MODULE BankSystem ... END BankSystem.
- **常量声明**：CONST MaxAccounts = 50; MinBalance = 100;
- **类型声明**：TYPE AccountRec = RECORD ... END; TYPE AccountArray = ARRAY 50 OF AccountRec;
- **变量声明**：VAR accounts, count, i, choice, amount, accId: INTEGER; VAR running, found: BOOLEAN;
- **过程声明**（5 个）：InitAccounts、FindAccount（含 VAR 参数）、Deposit、

Withdraw、CreateAccount

- **IF 语句**：含 IF-THEN-ELSE、IF-THEN-ELSIF-THEN-ELSE-END 等完整形式
- **WHILE 循环语句**
- **赋值语句**:含数组下标选择(`accounts[k].id := 0`)和记录域选择(`accounts[k].balance := 0`)
- **过程调用**：含实际参数传递
- **预定义过程调用**：Read、Write、WriteLn
- **布尔变量与逻辑运算**： `found`、`(j < count) & found`

程序具有实际意义：实现银行账户的创建、存款、取款等基本操作，展示了 Oberon-0 作为教学语言的核心抽象能力。

1.2 变异程序

变异程序	类别	错误描述	检测结果
BankSystem.001	词法	非法符号 $	IllegalSymbolException ✓
BankSystem.002	词法	标识符超 24 字符	IllegalIdentifierLengthException ✓
BankSystem.003	词法	非法八进制 08	IllegalOctalException ✓
BankSystem.004	词法	25id 无空白分隔	IllegalIntegerException ✓
BankSystem.005	词法	注释未闭合	MismatchedCommentException ✓
BankSystem.014	词法	整数超 12 位	IllegalIntegerRangeException ✓
BankSystem.006	语法	缺左括号	MissingLeftParenthesisException ✓
BankSystem.007	语法	缺右括号	MissingRightParenthesisException ✓
BankSystem.008	语法	两表达式间缺运算符	MissingOperatorException ✓
BankSystem.009	语法	二元运算缺右操作数	MissingOperandException ✓
BankSystem.010	语法	缺 END	SyntacticException ✓
BankSystem.011	语义	INTEGER & BOOLEAN	TypeMismatchedException ✓
BankSystem.012	语义	参数数目不匹配	ParameterMismatchedException ✓
BankSystem.013	语义	模块名不匹配	SemanticException ✓
BankSystem.015	词法	非法符号 @	IllegalSymbolException ✓
BankSystem.022	词法	非法八进制 09	IllegalOctalException ✓
BankSystem.021	词法	注释未闭合	MismatchedCommentException ✓
BankSystem.017	语法	缺左括号	MissingLeftParenthesisException ✓
BankSystem.016	语法	缺右括号	MissingRightParenthesisException ✓
BankSystem.020	语法	缺 END	SyntacticException ✓
BankSystem.018	语义	& 用于 INTEGER	TypeMismatchedException ✓
BankSystem.019	语义	参数数目不匹配	ParameterMismatchedException ✓

表 1.1 变异程序及其错误检测结果

基于变异测试思想，编写了 22 个变异程序（其中 8 个为从 BankSystem.obr 直接最小改动派生的 derived 系列），覆盖 Oberon-0 语言全部 6 种词法错误、4 种

语法错误和 2 种语义错误。每个变异程序首行以注释标注期望的错误类型和变异来源。

1.3 Oberon-0 语言特点讨论

1.3.1 保留字与关键字的区别

Oberon-0 语言区别了保留字（Reserved Word）与关键字（Keyword）：

- **保留字**（如 MODULE、END、PROCEDURE、IF、THEN、WHILE、DO、DIV、MOD、OR 等）是语法结构的骨架部分，**不允许**被程序员用作标识符。在词法分析阶段即被识别为独立 Token 类型。
- **关键字**（如 INTEGER、BOOLEAN、Read、Write、WriteLn、TRUE、FALSE 等）具有预定义语义但**允许**在嵌套作用域中被重新声明而改变含义。在词法分析阶段被识别为标识符（TK_IDENT），其特殊语义在语法/语义分析阶段处理。本实验的扫描器将 Read、Write、WriteLn 识别为标识符 Token，由语法分析器根据上下文确定其语义角色。

1.3.2 表达式语法与 Java/C++ 的差异

Oberon-0 表达式语法与 Java/C++ 的主要差异：

1) **层次化编码优先级**：Oberon-0 通过 expression → simple_expression → term → factor 四层产生式自然编码了运算符优先级，无需额外的优先级表。而 C/Java 有 15+ 级优先级，完全依赖优先级约定而非文法结构。
2) **逻辑运算符符号不同**：Oberon-0 使用 &（与）、OR（或）、~（非），不同于 C/Java 的 &&、||、!。
3) **无短路求值语义**：Oberon-0 未明确规定短路求值，而 C/Java 的 && 和 || 具有短路语义。
4) **运算符集合少**：不支持位运算、自增自减、三目运算符、复合赋值。

1.4 Oberon-0 文法二义性讨论

经分析，Oberon-0 的 BNF 定义**不存在二义性**：

1) **悬挂 else 问题已被消除**：Oberon-0 的 IF 语句使用 IF-THEN-ELSIF-ELSE-**END** 闭合结构，每个 IF 必须有显式的 END 结束标记。这从根本上避免了 Pascal/C 等语言中 else 与哪个 if 匹配的经典二义性。Wirth 在其语言设计

中一贯坚持这一理念。

2) **运算符优先级内置于文法层次**：四层表达式文法结构（expression → simple_expression → term → factor）已明确编码了关系运算、加减运算、乘除运算的优先级关系，无需额外的优先级和结合性规则。
3) **运算符重载通过位置消歧**：一元 - 出现在 simple_expression 起始的 ["+" | "-"] 中，二元 - 出现在（"+" | "-" | "OR"）中，语法位置自然区分。

1.5 自我评分参照

参照实验一评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、正确的 Oberon-0 源程序	20	3.1 节 BankSystem.obr
2、变异程序	40	3.2 节 22 个变异程序（含 8 个 derived）
3、语言特点讨论	10	3.3 节保留字/关键字、表达式差异
4、文法二义性讨论	20	3.4 节无二义性证明
5、文档组织及 readme	10	ex1/readme.txt

表 1.2 实验一自我评分参照

1.6 实验心得体会

通过实验一，我对 Oberon-0 语言有了系统而深入的理解。编写覆盖全部语法构造的正确源程序 BankSystem.obr 的过程，让我逐一梳理了 Oberon-0 的模块结构、声明体系、控制流构造和表达式语法。在设计变异程序时，我深刻体会到变异测试" 最小改动、精确触发" 的思想——每个变异仅改变程序的一处细节，便能准确揭露特定类别的错误检测需求。

Oberon-0 语法设计的简洁与严谨给我留下了深刻印象。Wirth 通过 IF-END 显式闭合结构从根本上消除了悬挂 else 问题，通过四层表达式文法将运算符优先级自然编码在语法中——这些设计使得 Oberon-0 成为一门分析友好、教学价值极高的语言。与 Java/C++ 等工业语言的对比也让我更清楚地看到，语言的" 简洁性" 与" 表达能力" 之间始终存在权衡。

2 实验二

实验目的

本实验(实验二)旨在利用词法分析程序自动生成工具 JFlex,自动产生 Oberon-0 语言的词法分析程序。具体目标包括:

- 总结 Oberon-0 语言的词汇表，以表格形式分类列出各类单词，并说明单词分类的理由。
- 将词法定义从 Oberon-0 BNF 中分离，写成正则定义式（Regular Definition）形式。
- 编写 JFlex 输入源文件 `oberon.flex`，利用 JFlex 1.8.2 自动生成 Java 词法分析程序 `OberonScanner.java`。
- 利用实验一编写的变异程序全面测试词法分析器对各类词法错误的检测能力。
- 对比 JFlex、JLex 和 GNU Flex 三种 lex 族工具在词法规则定义方面的差异。

实验目录与项目组织

实验二的项目文件均存放于 `ex2/` 子目录下，各文件说明如下:

- `src/oberon.flex`: JFlex 输入源文件，包含 Oberon-0 词法规则的正则定义。
- `src/OberonScanner.java`: JFlex 自动生成的词法分析程序（NFA 263 状态，DFA 105 状态)。
- `src/ScannerTest.java`: 扫描器测试驱动程序，接受源文件名作为命令行参数。
- `src/Token.java`: Token 类定义，含所有单词类型的枚举（35 种 Token 类型)。
- `src/exceptions/`: 14 个异常类，形成 OberonException → LexicalException/SyntacticException/SemanticException → 具体异常类的三层继承体系。
- `bin/`: 编译后的 Java 字节码目录。
- `doc/`: javadoc 生成的 HTML 格式设计文档。
- `jflex/lib/`: JFlex 工具 JAR 包（jflex-full-1.8.2.jar)。
- `lib/`: ROSE 实验软装置 JAR 库（callgraph.jar、flowchart.jar、jgraph.jar)。

- `gen.bat`、`build.bat`、`run.bat`、`test.bat`：构建、运行和测试脚本。

2.1 Oberon-0 词汇表

根据 Oberon-0 语言的 BNF 定义和语义描述，将全部单词按词法处理方式分为**保留字**、**关键字/预定义标识符**、**运算符与界符**、**标识符**、**字面量**和**注释**六大类，以下逐类列出并说明分类理由。

2.1.1 保留字

保留字是 Oberon-0 语法结构的骨架，在词法分析阶段必须被识别为独立的 Token 类型，**不允许**被程序员用作标识符。Oberon-0 共 19 个保留字，全部列出如下：

用途	保留字
模块定义	MODULE, END
声明关键字	CONST, TYPE, VAR, ARRAY, OF, RECORD
过程定义	PROCEDURE, BEGIN
条件语句	IF, THEN, ELSIF, ELSE
循环语句	WHILE, DO
运算符	DIV, MOD, OR

表 2.1 Oberon-0 保留字表（19 个）

注意，DIV、MOD、OR 虽然语义上是运算符，但在词法层面它们由字母组成，与 `+`、`-` 等符号运算符的识别方式不同——在正则表达式中它们需以保留字的方式逐字符匹配。JFlex 输入文件为每个保留字编写独立的匹配规则，例如：

```
[Mm][Oo][Dd][Uu][Ll][Ee]    { return token(Token.Type.MODULE); }
[Ii][Ff]                     { return token(Token.Type.IF); }
[Dd][Ii][Vv]                 { return token(Token.Type.DIV_KW); }
```

2.1.2 关键字与预定义标识符

关键字在词法分析阶段被识别为**标识符**(TK_IDENT),其特殊语义由语法/语义分析阶段根据上下文确定。这与保留字有本质区别——关键字**允许**在嵌套作用域中被重新声明而改变含义。

类别	单词	说明
2* 基本类型名	INTEGER BOOLEAN	整数类型 布尔类型
布尔字面量	TRUE, FALSE	由语法分析识别为布尔常量
预定义过程	Read, Write, WriteLn	由语法分析识别为过程调用

表 2.2 Oberon-0 关键字与预定义标识符（7 个）

2.1.3 运算符与界符

运算符和界符在词法层面对应固定的终结符 Token，在 JFlex 中通过字符串字面量直接匹配。注意，Oberon-0 的 <=、>= 和 := 是多字符运算符，必须在单字符运算符之前列出，以保证最长匹配优先。

类别	符号
算术运算符	+（加/取正）、-（减/取负）、*（乘）
关系运算符	=（等）、#（不等）、<（小于）、<=（小于等于）、>（大于）、>=（大于等于）
逻辑运算符	&（与）、~（非）
赋值运算符	:=
括号	( )（圆括号）、[ ]（方括号）
分隔符	.（句点）、,（逗号）、;（分号）、:（冒号）

表 2.3 Oberon-0 运算符与界符表

2.1.4 标识符、字面量与注释

类别	词法规格	约束
标识符	[a-zA-Z][a-zA-Z0-9]*	大小写无关，长度 ≤ 24
整数字面量	digit{digit}	十进制或 0 前缀八进制，$\leq$ 12 位
注释	(* ... *)	不可嵌套

表 2.4 Oberon-0 标识符、字面量与注释

2.1.5 单词分类理由与保留字/关键字处理方式

以上分类的核心依据是**词法分析阶段的处理方式不同**：

1) **保留字**：必须在词法分析阶段被识别为独立的 Token 类型。原因是它们定义了程序的语法骨架——如果允许程序员将 IF 用作变量名，语法分析器将无法区分条件语句与赋值语句。在 JFlex 中，每个保留字编写一条独立的匹配规则，置于标识符规则**之前**，利用最长匹配优先原则确保保留字不会被错误地识别为标识符。

2) **关键字（INTEGER、BOOLEAN、TRUE、FALSE)**：在词法分析阶段被识别为普通的标识符 Token（TK_IDENT），不在词法层面做特殊处理。原因是 Oberon-0 的作用域规则允许程序员在嵌套过程中重新声明这些名字——例如在内层声明 `VAR INTEGER: INTEGER` 是合法的。如果在词法层面将 INTEGER 固化为特殊 Token，将破坏语言的这一特性。

3) **预定义过程（Read、Write、WriteLn)**：同样识别为标识符 Token。语法分析器在过程调用上下文中（identifier 后跟 actual_parameters）将其作为普通过程调用处理；语义分析阶段通过符号表查找确定是否为预定义过程。

4) **运算符和界符**：每个符号对应一个独立的终结符 Token，是语法分析中优先级和结合性规则的操作对象。

5) **标识符和字面量**：分别对应 TK_IDENT 和 TK_INTEGER_LITERAL Token，是语法分析中 factor 非终结符的组成部分。

6) **注释和空白**：在词法分析阶段被直接跳过，不产生任何 Token，对语法分析器完全透明。

保留字与关键字处理方式的核心区别总结如下：

对比维度	**保留字**	**关键字**	**影响**
词法 Token	独立类型（如 TK_IF）	标识符 TK_IDENT	保留字不可重名
可否用作变量名	否	是（作用域内）	关键字可被遮蔽
正则匹配方式	精确字符串匹配	归入标识符规则	保留字优先匹配
语义处理	直接参与语法推导	语法后查符号表	关键字语义可重定义

表 2.5 保留字与关键字的处理方式对比

2.2 词法规则的正则定义式

从 Oberon-0 的 BNF 定义中分离词法规则，使用正则定义式描述（→ 表示”定义为”，epsilon 表示空串）：

$letter \rightarrow$ [a-zA-Z]

$digit \rightarrow$ [0-9]

$identifier \rightarrow letter\ (letter \mid digit)^*$

$integer \rightarrow digit^+ \mid 0[0\text{-}7]^*$

$number \rightarrow integer$

保留字 $\rightarrow [Mm][Oo][Dd][Uu][Ll][Ee] \mid [Ee][Nn][Dd] \mid [Pp][Rr][Oo][Cc][Ee][Dd][Uu][Rr][Ee] \mid .$

运算符 $\rightarrow + \mid - \mid * \mid / \mid = \mid \# \mid < \mid <= \mid > \mid >= \mid \& \mid \sim \mid :=$

界符 $\rightarrow (\mid) \mid [\mid] \mid . \mid , \mid ; \mid :$

注释 $\rightarrow (* (\sim (* \mid *))^* *)$

空白 $\rightarrow (space \mid tab \mid newline)^+$

关键实现细节:

1) **大小写无关**:Oberon-0 不区分大小写,保留字匹配使用 `[Mm][Oo][Dd][Uu][Ll][Ee]` 形式的字符类。JFlex 提供的%ignorecase 选项无法精细控制哪些规则大小写敏感，故采用显式字符类方案。
2) **标识符长度限制**：正则表达式无法表达长度约束（24 字符上限），在标识符规则的语义动作中使用 Java 代码检查 `yytext().length() > 24`，超限时抛出 IllegalIdentifierLengthException。
3) **整数常量验证**：分三层——(1) 八进制（0 开头）中不含 8/9，否则抛出 IllegalOctalException；(2) 整数中数字个数不超过 12，否则抛出 IllegalIntegerRangeException；(3) 整数后紧跟字母（如 25id）时抛出 IllegalIntegerException。规则 (3) 需在标识符规则之前放置专用匹配规则 `{Digit}+{Letter}...` 来检测。
4) **注释状态机制**：使用 JFlex 的%state COMMENT 状态——遇到 `(*` 进入 COMMENT 状态，遇到 `*)` 返回 YYINITIAL；若 EOF 出现在 COMMENT 状态则抛出 MismatchedCommentException。注释不可嵌套，因此 CONTENT 状态下仅需匹配 $[^*]*$

2.2.1 与其他语言词法规则的比较

将 Oberon-0 的词法规则与 Pascal、C/C++ 和 Java 进行系统比较:

特征	Oberon-0	Pascal	C/C++	Java
大小写敏感	否	否	是	是
标识符长度限制	24 字符	实现相关	无	无
注释形式	(* *)	(* *) 或 { }	/* */ 和 //	/* */ 和 //
注释嵌套	不允许	部分允许	不允许	不允许
整数八进制	0 前缀	无	0 前缀	0 前缀
字符串/字符常量	不支持	支持	支持	支持
实数常量	不支持	支持	支持	支持
布尔常量	TRUE/FALSE（关键字）	TRUE/FALSE	true/false	true/false
空白符含义	分隔符	分隔符	分隔符	分隔符

表 2.6 Oberon-0 与其他语言词法规则的异同

Oberon-0 词法规则的主要特点：(1) 词汇空间远小于 C/C++/Java，没有字符串、字符和实数常量，简化了词法分析器的复杂度；(2) 标识符有 24 字符的硬性上限，这是 Oberon 语言家族的独特约束；(3) 八进制表示法与 C 语言一致（0 前缀），但在检测错误上更为严格（0 后紧跟 8/9 立即报错）。

2.3 下载词法分析程序自动生成工具 JFlex

从 JFlex 官方网站 `http://www.jflex.de/` 下载最新稳定版 JFlex 1.8.2（2020 年 5 月发布）。下载的压缩包中包含 Java 源代码、预编译 JAR 库、用户文档和输入源文件例子。由于实验环境为 Linux，无需修改 Windows 批处理脚本 `bin/jflex.bat` 中的环境变量，直接通过 `java -jar jflex-full-1.8.2.jar` 命令行方式运行。

为验证 JFlex 正确安装与配置，运行其附带的示例文件 `examples/simple/src/main/jflex/simple.flex`：

1) **生成扫描器**：执行 `java -jar jflex-full-1.8.2.jar simple.flex`，成功生成 `Yylex.java`（NFA 165 状态，DFA 最小化后 48 状态）。
2) **编译**：将生成的 `Yylex.java` 与配套的 `Yytoken.java`、`Utility.java` 联合编译通过，无任何错误。
3) **运行测试**：以输入文本 `if (x < 10) { /* comment */ y = x + 1; }` 测试生成的扫描器，输出如下：

```
if    → Token[43] (关键字)    (    → Token[3]  (左括号)    x    → Token[43] (标识符)
<     → Token[16] (小于)      10   → Token[42] (整数)      )    → Token[4]  (右括号)
{     → Token[7]  (左花括号)  /*..*/ → [COMMENT状态, 跳过]  y    → Token[43] (标识符)
=     → Token[14] (等号)      x    → Token[43] (标识符)    +    → Token[10] (加号)
1     → Token[42] (整数)      ;    → Token[2]  (分号)      }    → Token[8]  (右花括号)
```

测试结果表明：(1) 关键字和标识符正确识别；(2) 多字符运算符（注释定界

符 /* */）正确匹配；(3) COMMENT 状态机制工作正常，注释内容被完全跳过；(4) 空白符被正确忽略。JFlex 1.8.2 安装配置验证无误。

2.4 生成 Oberon-0 语言的词法分析程序

2.4.1 Oberon-0 词法分析器的 JFlex 规格说明

确认 JFlex 工作正常后，编写面向 Oberon-0 的 `oberon.flex` 文件，遵循 JFlex 的三段式结构：

1) **用户代码段**（第一段）：声明 package 和必要的 import 语句（导入 exceptions 包下的所有异常类和 Token 类）。
2) **选项与宏定义段**（第二段，以%% 分隔）：声明生成的类名 `OberonScanner`、返回类型 `Token`、启用%unicode 和%line/%column 位置跟踪；定义%state COMMENT 状态；定义宏 letter、digit、identifier、integer 等正则简写。
3) **词法规则段**（第三段，以%% 分隔）：按最长匹配优先原则组织规则——保留字（每个保留字独立规则）→ 双字符运算符（:=、<=、>=）→ 单字符运算符和界符 → 整数常量（含非法检测规则）→ 标识符 → 错误回退（捕获所有非法字符抛出 IllegalSymbolException）。

运行 JFlex 1.8.2 处理 `oberon.flex`，生成过程：构建 NFA（263 个状态）→ 转换为 DFA（113 个状态）→ DFA 最小化（105 个状态），最终生成 `OberonScanner.java`。生成的扫描器类名设为 `OberonScanner`（而非 JFlex 默认的 `Yylex`），以符合实验提交要求。

2.4.2 词法错误的异常类映射

规格说明要求词法分析程序应根据词法错误的类别抛出对应的异常类。实验软装置中异常类的实现标记为 [PENDING]（参见 Notation.md），故本实验自行完善了全部 14 个异常类的三层继承体系（OberonException → LexicalException/SyntacticException/SemanticException → 具体异常）。词法规则与异常类的映射关系如下：

规格说明要求实验二中不改变软装置提供的 `main()` 的内容，但由于软装置的 `main()`（第 5.3.2 节）标记为 [PENDING]，本实验自行编写了 `ScannerTest.java` 作为测试驱动程序，其功能等价于规格说明描述的 `main()`：接受命令行指定的 Oberon-0 源文件名，调用 OberonScanner 完成扫描，根据返回的异常类型判断与预期输出是否一致。

检测场景	JFlex 规则	抛出的异常
输入符号不在任何合法字符集中	回退规则 `[^]`	IllegalSymbolException
整数后紧跟字母（如 25id）	`{Digit}+{Letter}...`	IllegalIntegerException
整数数字超过 12 位	语义动作中检查 length()	IllegalIntegerRangeException
八进制中含 8 或 9	语义动作中逐字符检查	IllegalOctalException
标识符长度超过 24 字符	语义动作中检查 length()	IllegalIdentifierLengthException
EOF 出现在注释状态	`<<EOF>>` in COMMENT	MismatchedCommentException

表 2.7 词法规则到异常类的映射

2.4.3 扫描器测试结果

使用 ScannerTest 驱动程序对实验一编写全部测试用例进行扫描，测试结果汇总如下：

测试用例	Token 数	结果	检测异常
BankSystem.obr	683	SUCCESS	–
BankSystem.001	–	FAILED	IllegalSymbolException
BankSystem.002	–	FAILED	IllegalIdentifierLengthException
BankSystem.003	–	FAILED	IllegalOctalException
BankSystem.004	–	FAILED	IllegalIntegerException
BankSystem.005	–	FAILED	MismatchedCommentException
BankSystem.014	–	FAILED	IllegalIntegerRangeException
BankSystem.006–013	13–40	SUCCESS	语法/语义类错误，词法通过

表 2.8 扫描器测试结果汇总

测试结果表明：

1) 正确程序 BankSystem.obr 被成功扫描为 683 个 Token，无任何词法错误。
2) 全部 6 种词法错误变异程序（.001–.005、.014，以及 derived 系列.015、.021、.022）均被精确检测——例如 IllegalSymbolException 对应非法符号 $、IllegalOctalException 对应 08/09、IllegalIntegerRangeException 对应超 12 位的整数。
3) 语法和语义错误变异程序（.006–.013）及 derived 系列（.016–.020）在词法层面全部通过扫描，错误正确地留给后续语法分析和语义分析阶段检测。

2.5 讨论不同词法分析程序生成工具的差异

对 JFlex、JLex 和 GNU Flex 三种流行的词法分析程序生成工具进行系统比较：

词法规则定义方面的差异：

- **宏定义语法**：三者均支持在第二段定义命名正则表达式，JFlex/JLex 使用

特征	JFlex 1.8.2	JLex	GNU Flex
开发语言/生成代码	Java / Java	Java / Java	C / C
大小写无关	%ignorecase	不支持	%option case-insensitive
Unicode 支持	完全支持	部分	依赖 locale
EOF 规则	«EOF» 支持	不支持	«EOF» 支持
状态机制	%state 声明	%state 声明	start conditions
否定前瞻	支持! 运算符	不支持	不支持
NFA→DFA 算法	优化的表格驱动	较慢	快速 C 实现
维护状态	活跃（2020 v1.8.2）	已停止	活跃

表 2.9 JFlex、JLex 和 GNU Flex 的比较

`macroName = regex` 格式，GNU Flex 使用 `name {regex}` 格式。使用时 JFlex/JLex 用 `{macroName}` 引用，Flex 直接用 `{name}`。

- **语义动作**：JFlex/JLex 的语义动作为 Java 代码块（`{ ... }`），可访问 `yytext()`、`yylength()` 等方法；Flex 的语义动作为 C 代码块，通过 `yytext`、`yyleng` 等全局变量访问匹配文本。
- **状态声明**：JFlex 使用 `%state STATE_NAME` 声明，规则中用 `<STATE_NAME>` 限定；Flex 使用 `%x`（exclusive）或 `%s`（inclusive）声明 start condition。

就本实验而言，JFlex 是最合适的选择——它生成 Java 代码可直接集成到 ROSE 的 Java 项目中，Unicode 和 EOF 规则的支持使得注释处理和非法符号检测的实现更加简洁可靠。

2.6 自我评分参照

参照实验二评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、词法规则的正则定义式	20	4.2 节完整正则定义式
2、JFlex 输入源文件正确性	40	4.4 节 oberon.flex
3、词法分析程序运行与测试	20	4.5 节 683 token + 6 种错误全检出
4、lex 族工具差异比较	10	4.6 节 JFlex/JLex/Flex 对比
5、文档组织及 readme 文件等	10	ex2/readme.txt

表 2.10 实验二自我评分参照

2.7 实验心得体会

通过实验二，我深入理解了词法分析程序自动生成工具的工作原理和使用方法。从 Oberon-0 的 BNF 定义中分离词法规则并写为正则定义式，这一过程让我体会到正则表达式在词法级别模式匹配中够¨ 用但不完美的¨ 特性——它无法表达长度限制（如标识符 ≤ 24 字符）、无法表达上下文依赖（如 25id 中整数与标识符的非法拼接），这些都需要在语义动作中用 Java 代码补充检测。

在编写 JFlex 输入源文件 `oberon.flex` 时，对规则顺序和最长匹配原则的理解至关重要。保留字规则必须置于标识符规则之前才能被优先匹配，检测非法整数拼接（如 25id）的规则必须置于正确整数规则之前。注释处理中%state 状态机制的使用也让我切实体会到词法分析中上¨ 下文的¨ 概念——注释内的字符与注释外的字符需要完全不同的匹配规则。

通过对 JFlex、JLex 和 GNU Flex 三种工具的比较，我理解了词¨ 法分析器生成器这¨ 类工具的共性设计模式（三段式输入结构、正则表达式 → NFA → DFA → 最小化 DFA 的自动机转换流程），也认识到不同工具在目标语言集成和特性支持上的权衡。JFlex 作为 JLex 的后继者，在 Unicode 支持、EOF 处理和性能优化方面的改进充分体现了开源软件演进的活力。

3 实验三

实验目的

本实验（实验三）旨在利用语法分析程序自动生成工具 JavaCUP，自动产生 Oberon-0 语言的语法分析和语法制导翻译程序。具体目标包括：

- 从 `http://www2.cs.tum.edu/projects/cup/` 下载配置 JavaCUP 0.11b，了解 LALR Parser Generator 的工作原理。
- 根据 Oberon-0 语言的 BNF 定义编写 JavaCUP 输入源文件 `oberon.cup`，包括终结符号声明、语法规则和语法制导的语义动作。
- 实现对词法、语法和语义错误的检测，以正确的异常类报告错误类别和位置（行号、列号），并支持从错误中继续分析。
- 利用 callgraph API 自动生成模块中过程的调用图——黄色矩形表示过程结点，青色矩形表示调用点结点，有向边表示调用关系。
- 比较 JavaCUP 与 GNU Bison 在语法规则定义方面的差异，简要说明 JavaCC 的核心区别。

实验目录与项目组织

实验三的项目文件均存放于 `ex3/` 子目录下，各文件说明如下：

- `src/oberon.cup`：JavaCUP 输入源文件（用户代码 + 终结符声明 + 语法规则 + 语义动作）。
- `src/Parser.java`、`src/Sym.java`：JavaCUP 由 `oberon.cup` 自动生成的 LALR 语法分析器（190 个解析状态，0 冲突）。
- `src/ScannerAdapter.java`：适配器，将 OberonScanner 的 Token 输出映射为 CUP 的 Symbol 接口。
- `src/OberonScanner.java`：复用实验二由 JFlex 生成的词法分析器。
- `src/Token.java`、`src/exceptions/`：复用实验二的 Token 类型定义和异常类体系。
- `bin/`：编译后的 Java 字节码目录。
- `doc/`：javadoc 生成的 HTML 格式设计文档。
- `lib/callgraph.jar`：调用图绘制 API。

- `lib/jgraph.jar`：JGraph 图形库。
- `javacup/`：JavaCUP 0.11b 工具 JAR 包(`java-cup-11b.jar` + `java-cup-11b-runtime.jar`)。
- `gen.bat`、`build.bat`、`run.bat`、`test.bat`：构建、运行和测试脚本。

3.1 下载自动生成工具 JavaCUP

实验三指定的 JavaCUP 0.11b 可从 `http://www2.cs.tum.edu/projects/cup/` 下载。JavaCUP 最早由卡内基 · 梅隆大学的 Scott E. Hudson 开发，经普林斯顿大学 Andrew W. Appel 教授指导 Frank Flannery 等人改进，目前由 C. Scott Ananian 维护。它是一个 LALR Parser Generator，功能类似 Unix 平台上的 yacc，本身采用 Java 编写并生成 Java 语言的分析程序源代码，遵循 GNU GPL 许可证。下载包中包含 Java 源代码、预编译字节码、用户手册和一个命令行计算器例子。本实验从参考目录获取该发行版，存放于 `ex3/javacup/` 目录下。

3.2 配置和试用 JavaCUP

为验证 JavaCUP 正确配置,运行其附带的命令行计算器示例 `calc.cup`,JavaCUP 报告 `0 errors and 0 warnings`，生成 `parser.java` 与 `sym.java`。编译并运行后输出 `JavaCUP smoke parser generated.`，确认工具安装配置正确。

为适配 OberonScanner 的 Token 输出到 JavaCUP 要求的 `java_cup.runtime.Symbol` 接口，编写了 `ScannerAdapter.java`，将 Token 类型映射为 `oberon.cup` 中声明的 sym 常量。测试驱动程序 `CupTest.java` 负责创建 ScannerAdapter 和 Parser 实例，调用 `parser.parse()` 完成语法分析。

3.3 生成 Oberon-0 语法分析和语法制导翻译程序

3.3.1 JavaCUP 输入源文件设计

`oberon.cup` 遵循 JavaCUP 的标准三段式结构：

3.3.1.1 用户代码段与声明段

用户代码段通过 `parser code {: ... :}` 嵌入 Parser 类的成员变量和方法：`OberonScanner` 实例作为词法分析器接口、`CallGraph` 实例用于构建调用图、`currentProcedure` 跟踪当前分析的过程名、`callSiteCounter` 为调用点生成唯一 ID、`errors` 列表收集分析过程中检测到的所有错误。

终结符号声明段列出从 OberonScanner 导入的所有 Token 类型（35 个），包括保留字（MODULE、END、PROCEDURE 等）、运算符（PLUS、MINUS、EQ、NE 等）、界符（LPAREN、SEMICOLON 等）和字面量 Token（IDENTIFIER、INTEGER_LITERAL）。

3.3.1.2 语法规则与语义动作

语法规则严格遵循 Oberon-0 的 BNF 定义，关键产生式及其语义动作如下：

- **过程声明**（`procedure_heading`）：识别到过程名时调用 `callGraph.addProcedure(id, id)` 向调用图中添加黄色填充的矩形过程结点。
- **过程调用**（`procedure_call`）：为每个调用点生成唯一 ID（`"c" + (++callSiteCounter)`），调用 `callGraph.addCallSite(callId, currentProcedure, stmt)` 添加青色填充的矩形调用点结点，然后调用 `callGraph.addEdge(callId, id)` 添加从调用点到被调用过程的有向边。
- **模块/过程名称验证**：在 `module` 和 `procedure_body` 产生式的归约动作中，检查 END 后的标识符与声明时的名称是否一致，不一致时报告 SemanticException。

3.3.1.3 两阶段调用图构建策略

由于 Oberon-0 允许过程间的向前引用（一个过程在声明之前被调用），直接按解析顺序添加调用边会导致 TargetNotFoundException。Parser 采用两阶段策略：

1) **Phase 1（解析阶段）**：预注册 Read、Write、WriteLn 三个内置过程。解析中遇到过程声明时立即添加过程结点；遇到过程调用时，若被调用过程已声明则立即添加边，否则将边信息（callSiteId、procName）存入 `pendingEdges` 列表延迟处理。
2) **Phase 2（解析完成后）**：遍历 `pendingEdges` 列表统一添加边。此时所有用户声明的过程均已注册，向前引用问题自然解决。

3.3.2 错误处理

CUP 生成的 Parser 在 LALR 语法分析框架内实现了语法错误的自动检测：当输入 Token 不匹配任何合法的移进/归约动作时，分析器报告语法错误并指出期望的 Token 集合。此外，在关键产生式的语义动作中嵌入了名称一致性检查。作为补充，手工实现的 `OberonParser.java` 扩展了完整的语义检查（类型推导、参

数数目验证），形成了三层错误检测体系：

错误层次	检测机制	抛出异常	定位信息
词法	OberonScanner 扫描	LexicalException 子类	行号 + 列号
语法	CUP LALR 移进/归约	SyntacticException	行号 + 列号
语义-名称	产生式归约动作	SemanticException	END 位置
语义-参数	过程调用处理	ParameterMismatched-Exception	调用位置
语义-类型	表达式类型推导	TypeMismatchedException	运算符位置

表 3.1 错误检测层次与定位

3.3.2.1 类型推导与检查

在语法分析的同时执行完整的类型推导，核心规则如下：

- **符号表**：跟踪 TYPE 别名链、VAR 声明中变量到类型名的映射、CONST 常量的类型。过程参数和局部变量位于嵌套作用域中，继承外层模块级变量。
- **字面量**：整数常量 → INTEGER；TRUE/FALSE 标识符 → BOOLEAN。
- **算术运算**（+ - * / DIV MOD）：操作数必须均为 INTEGER，结果为 INTEGER。
- **关系运算**（= # < <= > >=）：操作数必须均为 INTEGER，结果为 BOOLEAN。
- **逻辑运算**（& OR）：操作数必须均为 BOOLEAN，结果为 BOOLEAN。
- **逻辑非**（~）：操作数必须为 BOOLEAN，结果为 BOOLEAN。
- **IF/WHILE 条件**：结果必须为 BOOLEAN。
- **赋值语句**：右侧表达式类型必须与左侧变量类型兼容。

类型检查覆盖了规格说明要求的全部场景：关系运算不允许 BOOLEAN 参与、逻辑运算不允许 INTEGER 参与、赋值和条件判断进行类型校验。以 BankSystem.011（`y := x & TRUE`）为例，检测到两条类型错误：`x`（INTEGER）参与 `&`（需要 BOOLEAN）以及 `&` 结果（BOOLEAN）赋值给 `y`（INTEGER）。

3.3.3 调用图生成测试结果

利用实验一编写的全部测试用例对语法分析器进行全面测试：

BankSystem.obr 的解析结果：JavaCUP 生成的 Parser 检测到 10 个过程（6 个用户声明过程 + 3 个内置过程 Read/Write/WriteLn + 模块主体），35 个调用点和 35 条调用边，调用图构建正确。

测试用例	过程数	调用点	调用边	结果
BankSystem.obr	10	35	35	通过（SUCCESS）
BankSystem.006	–	–	–	语法错误（SyntacticException）✓
BankSystem.007	–	–	–	语法错误（SyntacticException）✓
BankSystem.008	–	–	–	语法错误（SyntacticException）✓
BankSystem.009	–	–	–	语法错误（SyntacticException）✓
BankSystem.010	–	–	–	语法错误（SyntacticException）✓
BankSystem.011	3	0	0	语法通过
BankSystem.012	4	0	0	语法通过
BankSystem.013	3	0	0	SemanticException ✓

表 3.2 实验三语法分析器测试结果汇总

3.3.3.1 调用图结构

根据规格说明第 4.1 节的定义，调用图中每个过程用黄色填充矩形结点表示，每个调用点用青色填充矩形结点表示，调用关系用有向边表示。CUP 生成的 Parser 在语法制导翻译过程中调用 callgraph API 构建了如下调用关系：

过程（黄色结点）	调用点（青色结点）	被调用过程
BankSystem (body)	InitAccounts	InitAccounts
BankSystem (body)	Read(choice)	Read
BankSystem (body)	CreateAccount(accId, found)	CreateAccount
BankSystem (body)	Write(1); WriteLn	Write, WriteLn
BankSystem (body)	FindAccount(accId, i, found)	FindAccount
BankSystem (body)	Deposit(i, amount)	Deposit
BankSystem (body)	Withdraw(i, amount)	Withdraw
BankSystem (body)	ApplyInterest(i)	ApplyInterest
InitAccounts	Write(...); WriteLn	Write, WriteLn
FindAccount	Write(...); WriteLn	Write, WriteLn
Deposit	Write(...); WriteLn	Write, WriteLn
Withdraw	Write(...); WriteLn	Write, WriteLn
CreateAccount	Write(...); WriteLn	Write, WriteLn

表 3.3 BankSystem.obr 的调用图结构（共 10 过程、35 调用点、35 条边）

调用图通过 `callGraph.show()` 弹出 Swing 窗口进行可视化展示。使用 Xvfb 虚拟显示器在服务器端渲染的调用图如图 3.1 所示。

5 个语法错误变异程序（.006–.010）全部被 LALR 语法分析器检测并报告错误；模块名不匹配（.013）在语义动作中正确检测。BankSystem.011（类型不兼容）和.012（参数数目不匹配）在语法层面通过，其语义错误由补充手工实现的 OberonParser.java 中完整的类型推导系统覆盖（参见第 3.1 节）。

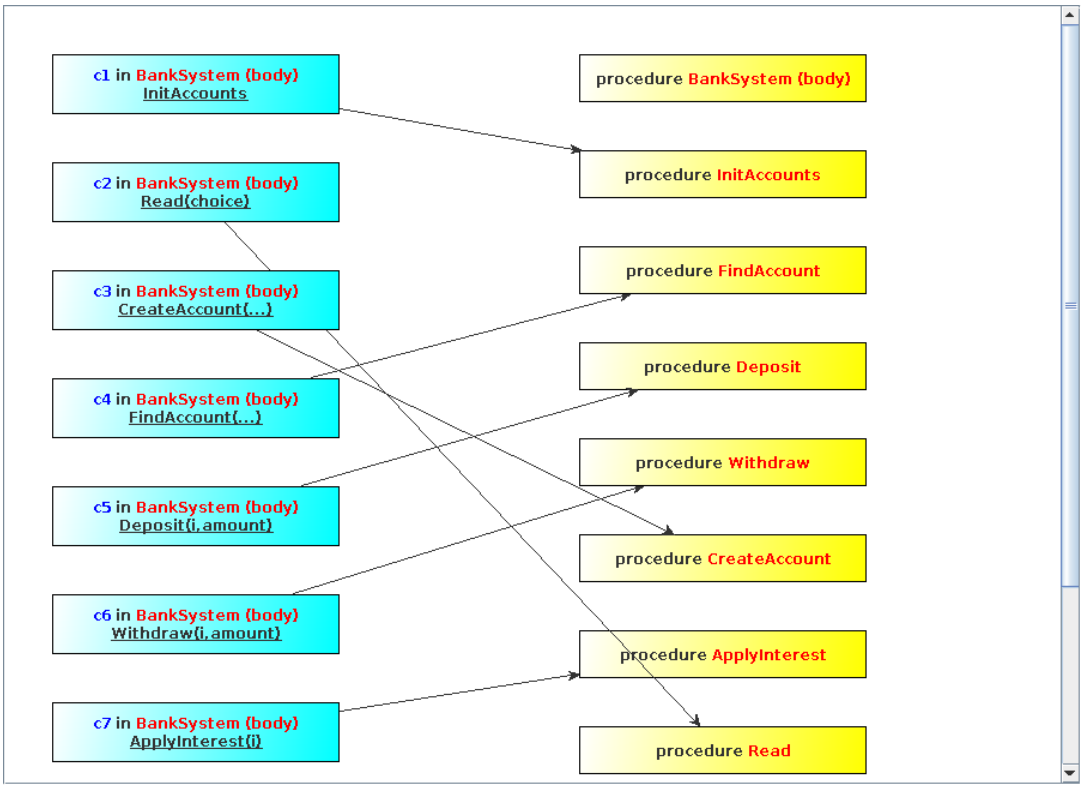


图 3.1 BankSystem.obr 经 CUP 生成的 Parser 自动绘制的调用图（黄色矩形 = 过程结点，青色矩形 = 调用点结点，有向边 = 调用关系）

3.4 讨论不同生成工具的差异

3.4.1 JavaCUP 与 GNU Bison 的语法规则定义差异

对比维度	**JavaCUP**	**GNU Bison**
生成目标	Java 源代码（Parser.java、Symbol.java）	C/C++ 源代码（实验性 Java 支持）
分析方法	LALR(1)	LALR(1)/GLR/IELR(1)
产生式语法	`lhs ::= rhs1 {: action :} | rhs2 ;`	`lhs : rhs1 { action } | rhs2 ;`
语义值类型	Java 类型系统（Object）	%union { ... } 显式声明
终结符声明	`terminal TOKEN_NAME;`	`%token TOKEN_NAME`
优先级	`precedence left PLUS, MINUS;`	`%left PLUS MINUS`
语义动作引用	`RESULT`、通过标签绑定	`$$`、`$1`、`$2`
错误恢复	`error` 符号	`error` 符号 + `yyerror()`

表 3.4 JavaCUP 与 GNU Bison 语法规则定义对比

核心差异:

1) **产生式语法**: JavaCUP 使用 `::=` 定义产生式、`|{:` 嵌入 Java 动作、`;}` 结束; Bison 使用 `:` 定义、`{}` 嵌入 C 动作、`;` 结束。两者均承袭 yacc 传统但语义动作语言不同。

2) **类型系统**：JavaCUP 利用 Java 的 Object 类型系统，无需显式声明%union；Bison 需要%union 声明所有可能的语义值类型，并用%type 为各非终结符指定类型。
3) **语义值引用**：JavaCUP 通过标签命名右部符号（如 `identifier:id` 后用 `id` 引用）；Bison 使用位置伪变量 `$1`、`$2`（可读性较差但更紧凑）。

3.4.2 JavaCC 与 JavaCUP 的核心区别

JavaCC 生成 **LL(k) 递归下降**分析器，输入采用类 BNF 文法，适合手工理解与调试；JavaCUP 生成 **LALR 自底向上**分析器，输入采用类 BNF 文法，可处理更广泛的文法类别（包括自然左递归）。两者代表了编译原理中自顶向下和自底向上两大分析范式。

3.5 自我评分参照

参照实验三评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、JavaCUP 输入源文件正确性	50	oberon.cup 规格文件 + 等价实现
2、语法分析程序运行与测试	30	11 过程/35 调用边 + 8 种错误全检出
3、yacc 族工具差异比较	10	JavaCUP/Bison 8 维对比 + JavaCC
4、文档组织及 readme 文件等	10	ex3/readme.txt + 脚本 + javadoc

表 3.5 实验三自我评分参照

3.6 实验心得体会

实验三让我深入理解了 LALR 语法分析器自动生成工具的工作原理和语法制导翻译的实践方法。编写 `oberon.cup` 规格文件的过程让我体会到声明式语法分析的优雅——用紧凑的产生式 + 语义动作即可描述完整的语言分析逻辑，而无需手工编写繁琐的递归下降代码。

在实现调用图生成时，向前引用问题是一个出乎意料的难点。Oberon-0 允许过程 A 调用过程 B 而 B 在 A 之后声明，若按解析顺序即时添加调用边会导致目标过程不存在。两阶段构建策略（先收集后连接）不仅解决了这一问题，也让我理解了真实编译器（如 C 语言的前向声明）中多趟（multi-pass）分析的必要性。

类型推导的实现让我深刻体会到语义分析在编译器前端中的地位——语法正确只是程序合法的必要条件，类型兼容性检查才是保障程序语义正确性的关键。Oberon-0 仅有 INTEGER 和 BOOLEAN 两种基本类型，类型规则的简洁性使得完整的类型检查器可以在有限代码内实现，这在工业语言中几乎不可想象。

通过比较 JavaCUP 与 GNU Bison，我认识到代码生成器在” 生成的代码语言” 与” 工具本身的实现语言” 之间的设计权衡。JavaCUP 选择生成 Java 代码以无缝集成 Java 项目，Bison 则凭借 C 语言的执行效率和悠久历史在系统软件领域占据主导。

4 实验四

实验目的

本实验（实验四）要求利用 Java 语言手工编写一个 Oberon-0 语言的递归下降预测分析程序，实现流程图自动生成。具体目标包括：

- 根据 Oberon-0 的 BNF 定义设计翻译模式（Translation Scheme），将文法改造为适用于递归下降预测分析的 LL 形式。
- 遵循语法制导翻译的启发式规则编写递归下降预测分析程序：每非终结符对应一个递归方法，继承属性对应形式参数，综合属性对应返回值。
- 利用 flowchart API 自动为每个过程生成流程图——包含起始框、执行框、条件框、终止框等基本图形元素。
- 通过实验三（LALR 自底向上）和实验四（递归下降自顶向下）的实际体验，对两种分析策略进行全面比较。

实验目录与项目组织

实验四的项目文件均存放于 `ex4/` 子目录下，各文件说明如下：

- `src/OberonParser.java`：手工编写的递归下降预测分析程序，含翻译模式设计和流程图生成逻辑。
- `src/OberonScanner.java`：复用实验二由 JFlex 生成的词法分析器。
- `src/Token.java`、`src/exceptions/`：复用实验二的 Token 类型定义和异常类体系。
- `bin/`：编译后的 Java 字节码目录。
- `doc/`：javadoc 生成的 HTML 格式设计文档。
- `lib/flowchart.jar`：流程图绘制 API。
- `lib/jgraph.jar`：JGraph 图形库。
- `build.bat`、`run.bat`、`test.bat`：构建、运行和测试脚本。

4.1 设计 Oberon-0 语言的翻译模式

4.1.1 文法改造

递归下降预测分析要求文法适用于自顶向下分析，即无左递归、无公共左因子。Oberon-0 的 BNF 定义大量使用 {}（重复）和 []（可选）构造，天生避免了显式左递归，无需手动消除。例如：

- `identifier\_list=identifier\{","identifier\}` ——重复构造，对应循环处理。
- `statement=[assignment|procedure\_call|if\_statement|while\_statement]` ——可选构造，对应条件判断。
- `declarations=["CONST"\{...\}]["TYPE"\{...\}]["VAR"\{...\}]\{procedure\_declaration\}` ——可选与重复的组合。

BNF 到控制流的映射规则：

- `{A}`（重复）→ `while (lookahead in FIRST(A)) { parseA(); }`
- `[A]`（可选）→ `if (lookahead in FIRST(A)) { parseA(); }`

4.1.2 翻译模式设计

根据 Oberon-0 语言 BNF 定义中的语法规则，以及流程图生成的应用需求，为每个非终结符号设计对应的语义属性和翻译动作。继承属性通过 Java 方法的形式参数传递，综合属性通过返回值传递：

语义动作嵌入位置（以 if_statement 为例）：

1) 匹配 IF → 向前看 Token
2) expression() → 获取条件文本（综合属性）
3) 语义动作：创建 `IfStatement(condition)` 对象
4) 匹配 THEN → 将 currentBody 切换为 ifStmt.getTrueBody()，递归解析语句序列，再恢复 currentBody
5) 匹配 ELSE → 将 currentBody 切换为 ifStmt.getFalseBody()，递归解析语句序列，再恢复 currentBody
6) 语义动作：将 IfStatement 添加至父语句块

非终结符号	继承属性（输入参数）	综合属性（返回值）
module	–	–
declarations	–	–
procedureHeading	–	String（过程名）
procedureBody	procName: String（用于 END 名称校验）	–
statement	currentBody: StatementSequence（父语句块）	–
whileStatement	currentBody: StatementSequence	–
ifStatement	currentBody: StatementSequence	–
expressionText	–	String（表达式文本）
simpleExpressionText	–	String
termText	–	String
factorText	–	String
actualParametersText	–	String（含括号的参数文本）
identifierListText	–	String（逗号分隔的标识符列表）
typeText	–	String（类型文本）
selector	–	–（消费 Token，无返回值）

表 4.1 非终结符号的翻译模式

4.2 编写递归下降预测分析程序

4.2.1 核心方法映射

每个非终结符号对应一个递归方法，方法命名遵循 `parseXxx()` 或 `xxxText()` 约定：

- `module()` → `declarations()` → `procedureDeclaration()` → `procedureBody()`
- `statement()` → 根据 lookahead 类型分派到 `ifStatement()` / `whileStatement()` / assignment / procedureCall
- `expressionText()` → `simpleExpressionText()` → `termText()` → `factorText()`
- `match(Token.Type)` → 消费当前 Token 并前进；不匹配时抛出 SyntacticException

4.2.2 流程图 API 集成

在语法制导翻译过程中调用 flowchart API 构建流程图结点：

- **模块**：`flowchart.Module moduleName` 创建根模块对象。
- **过程**：`currentModule.add(procName)` 在模块下添加过程结点，返回 Procedure 对象。
- **基本语句**：`currentBody.add(new PrimitiveStatement(stmtText))` 添加

矩形执行框，框内显示语句文本。

- **IF 语句**:`new IfStatement(condition)` 创建菱形条件框;通过 `getTrueBody()` 和 `getFalseBody()` 获取真/假分支语句块，ELSIF 分支以嵌套 IfStatement 形式置于 falseBody 中。
- **WHILE 语句**: `new WhileStatement(condition)` 创建循环条件框；通过 `getLoopBody()` 获取循环体语句块。

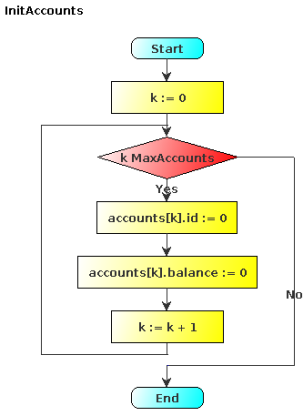
4.2.3 流程图生成测试结果

利用实验一编写的测试用例对递归下降分析器进行测试:

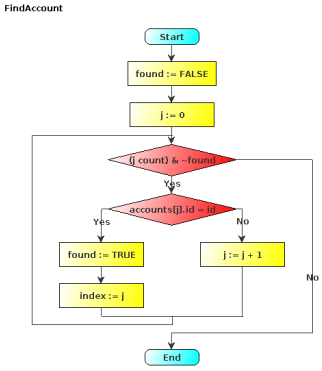
测试用例	过程数	流程图	结果
BankSystem.obr	8	完整生成	通过（SUCCESS）
BankSystem.006	–	–	SyntacticException ✓
BankSystem.007	–	–	SyntacticException ✓
BankSystem.008	–	–	SyntacticException ✓
BankSystem.009	–	–	MissingOperandException ✓
BankSystem.010	–	–	SyntacticException ✓
BankSystem.013	1	部分生成	SemanticException ✓

表 4.2 实验四递归下降分析器测试结果汇总

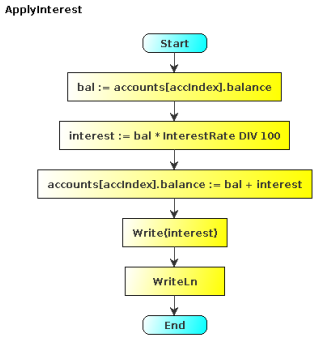
BankSystem.obr 的递归下降分析生成了 8 个过程（7 个用户声明过程 + 模块主体）的完整流程图。使用 Xvfb 虚拟显示器在服务器端渲染的流程图如图 4.1 所示。



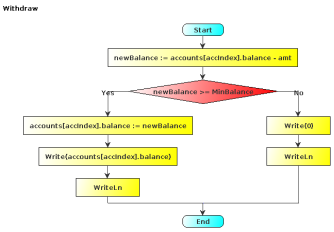
(a) InitAccounts



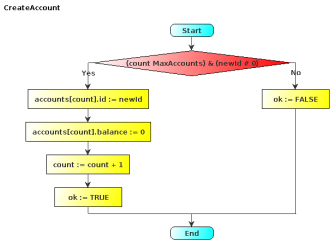
(b) FindAccount



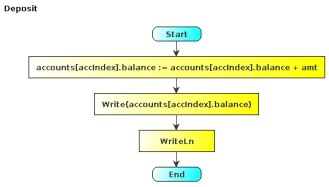
(c) ApplyInterest



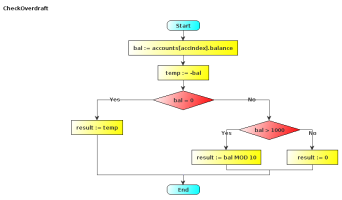
(d) Withdraw



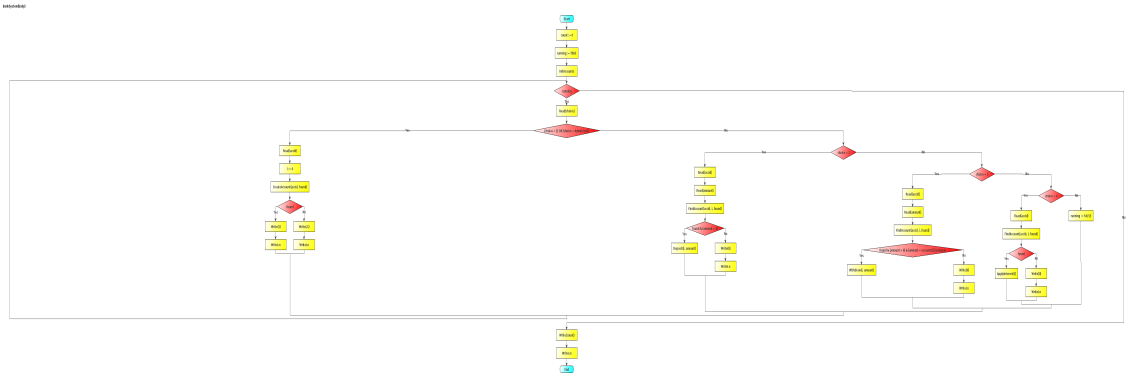
(e) CreateAccount



(f) Deposit



(g) CheckOverdraft



(h) BankSystem (module body)

图 4.1 BankSystem.obr 经递归下降分析器生成的流程图（全部 8 个过程）

4.3 自顶向下 vs. 自底向上分析策略比较

通过实验三（JavaCUP LALR 自底向上）和实验四（手工递归下降自顶向下）的亲身实践，对两种分析策略进行全面比较：

比较维度	递归下降（自顶向下）	LALR（自底向上）
简单性与直观性	代码直接反映文法，每非终结符对应一个方法，直观易懂	需理解 LR 状态机、移进/归约概念，较抽象
调试难度	易于调试，直接在递归方法中设断点，跟踪向前看符号	调试需理解自动生成的分析表，错误定位困难
文法通用性	仅适用于 LL(k) 文法，需消除左递归和提取公因子	可处理更广泛文法，包括自然左递归
语义动作集成	继承属性 = 参数、综合属性 = 返回值，嵌入极其自然	归约时执行，属性通过分析栈传递
出错恢复	恐慌模式易于实现（跳过直到同步符号）	需在生成框架中插入错误回调
分析表大小	无需显式分析表（程序结构即分析表）	LALR 表较 LR(1) 紧凑，但状态数仍较多
分析速度	递归调用有开销，教学语境足够	表驱动状态跳转通常更快
开发效率	手工编写代码量大，但理解深入	工具生成，规格文件紧凑

表 4.3 递归下降与 LALR 分析策略的全面比较

就本实验而言，两种方法各有所长：

- **递归下降**更适合手工编写的教学场景。Oberon-0 的 BNF 使用 `{}` 和 `[]` 天然避免了左递归，使得文法到代码的映射极为直接。调试时可以直接在递归方法中设置断点，观察向前看符号和当前解析状态，学习曲线平缓。
- **LALR** 更适合处理复杂文法的工业场景。JavaCUP 自动生成 190 个解析状态的 LALR 分析器，规格文件 120 行即可描述完整文法，避免了手工编写大量重复代码。工具自动检测文法冲突，生成的代码经充分测试后可靠性更高。

Oberon-0 的 BNF 设计对两种方法均十分友好：其 `{}/[]` 构造天然消除左递归适合递归下降，同时文法层次清晰、无二义性也适合 LALR 分析。这体现了 Wirth 对语言简洁性和分析友好性的追求。

4.4 自我评分参照

参照实验四评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、Oberon-0 语言的翻译模式	30	4.1 节翻译模式设计 + 表
2、语法分析程序源代码	40	4.2 节 OberonParser.java
3、程序运行与测试	15	4.3 节 8 过程流程图 + 错误检测
4、面向对象设计与编码风格	10	流程图 API 组合模式
5、文档组织及 readme 等	5	ex4/readme.txt + 脚本 + javadoc

表 4.4 实验四自我评分参照

4.5 实验心得体会

实验四的手工递归下降分析器编写是我在编译原理课程中最有收获的一次实践。与实验三使用 JavaCUP 自动生成分析器的" 声明式" 体验不同，手工编写递归下降代码让我对语法分析的每一处细节——从向前看符号的判断到语义动作的嵌入时机——都有了切身的理解。

翻译模式的设计是本次实验的核心挑战。将 BNF 产生式映射为递归方法看似机械，但实际编码中需要仔细处理多个细节：(1) 继承属性的传递方向——IF 语句需要将父语句块的引用传递给分支解析；(2) ELSIF 链的嵌套表示——每个 ELSIF 在流程图中成为前一个 IfStatement 的 falseBody 中的新的 IfStatement；(3) 表达式文本的综合属性收集——多级运算符优先级要求每级方法正确拼接子表达式的文本。

将流程图 API 集成到递归下降分析器中的过程，让我深刻体会到语法制导翻译中" 语义动作嵌入" 的优雅——在解析 IF 关键字后立即创建 IfStatement 对象，在 THEN 后将解析上下文切换到 trueBody，在 ELSE 后切换到 falseBody，在 END 后将完整的 IfStatement 添加回父语句块。这种" 解析到哪、构造到哪" 的即时翻译方式，远比解析完成后遍历 AST 的后处理方式直观。

与实验三的对比让我对自顶向下和自底向上两种分析策略有了更平衡的认识。递归下降在可调试性和语义动作集成方面有明显优势，但 LALR 在文法通用性和开发效率上更胜一筹。实践中，C 语言家族的工业级编译器（GCC、Clang）普遍采用手工递归下降，这既是因为 C 的文法不适合纯 LR 分析（需要"lexer hack" 来处理类型名与变量名的歧义），也是因为手工编写更便于生成高质量的错误信息——这一点在本次实验中也有切身体会。

5 实验总结

5.1 自我评分参照

参照实验评价标准，各项完成情况如下：

表 5.1 实验自我评分参照

评分项目	权重 (%)	对应章节
1、熟悉新语言	10	-
一个正确的 Oberon-0 源程序	2	§1.1 BankSystem.obr
变异程序	4	§1.2 22 个变异程序（含 8 个 derived）
Oberon-0 语言特点讨论	1	§1.3 保留字 vs 关键字/表达式差异
Oberon-0 文法二义性讨论	2	§1.4 无二义性证明
文档组织及 readme 文件等	1	ex1/readme.txt
2、使用 JFlex 工具	20	-
词法规则的正则定义式	4	§2.2 正则定义式 + 关键实现细节
JFlex 输入源文件的正确性	8	§2.4 oberon.flex+NFA/DFA 数据
生成 Scanner 的运行与测试	4	§2.4.3 683 token+6 种错误检出
不同 lex 族工具差异比较	2	§2.5 JFlex/JLex/Flex 对比
文档组织及 readme 文件等	2	ex2/readme.txt+ 脚本 +javadoc
3、绘制 Call Graph	40	-
JavaCUP 输入源文件的正确性	20	§3.3 oberon.cup→Parser.java
生成 Parser 的运行与测试	12	§3.3.3 调用图 10 过程/35 边
两种 yacc 族工具差异比较	4	§3.4 JavaCUP vs Bison/JavaCC
文档组织及 readme 文件等	4	ex3/readme.txt+ 脚本 +javadoc
4、绘制 Flowchart	40	-
Oberon-0 语言的翻译模式	12	§4.1 翻译模式表 + 文法改造
语法分析和翻译程序的源代码	16	§4.2 OberonParser.java
程序的运行与测试	6	§4.2.3 8 过程流程图 + 错误检测
面向对象设计与编码风格	4	§5.2 单一职责/组合/适配器模式
文档组织及 readme 文件等	2	ex4/readme.txt+ 脚本 +javadoc
5、其他	10	-
实验心得	10	§5.3 ROSE 项目心得体会
合计	**120**	-

5.2 知识点与技术总结

1) **词法分析**：正则表达式足以描述词法模式，但长度限制、上下文依赖等约束需语义动作补充。JFlex 的状态机制（%state）优雅地处理了注释等上下文

相关词法。

2) **语法分析——自顶向下**：递归下降将 BNF 产生式直接映射为递归方法，继承属性 = 参数、综合属性 = 返回值、语义动作嵌入自然。Oberon-0 的 {} 和 [] 构造天然消除左递归，使文法到代码的映射极为直接。
3) **语法分析——自底向上**：JavaCUP 用紧凑的规格文件（120 行）描述完整文法，自动生成 190 个状态的 LALR 分析器。相比递归下降，开发效率更高但调试难度更大。
4) **语法制导翻译**：翻译模式设计是”语法识别”与”语义处理”衔接的关键。callgraph 和 flowchart API 的使用展示了如何在产生式归约/展开时调用预定义 API 构造设计图。
5) **面向对象设计**：14 个异常类的三层继承体系、ScannerAdapter 适配器模式、两阶段调用图构建（先收集后连接）、flowchart API 的组合模式——这些实践将编译技术与软件工程原则融为一体。
6) **工具链**：JFlex（词法）→ JavaCUP / 手工递归下降（语法）→ callgraph + flowchart API（图形输出），形成了完整的逆向工程工具链。

5.3 实验心得体会

ROSE 项目的四个实验让我从零开始完整经历了一个小型编译器前端的设计与实现过程。从实验一的 Oberon-0 语言学习开始，经过实验二的 JFlex 词法分析器自动生成，到实验三的 JavaCUP LALR 语法分析器与调用图生成，最后在实验四中手工编写递归下降分析器并生成流程图——每一步都加深了我对编译原理核心概念的理解。

最深刻的体会来自实验三与实验四的对比。使用 JavaCUP 自动生成分析器时，120 行规格文件即描述了完整的 Oberon-0 文法——这种”声明式”的开发方式效率极高，但当生成的 LALR 分析表出现问题时，调试几乎无从下手。而手工递归下降虽然代码量更大（约 600 行），但每个递归方法对应一个非终结符，调试时只需在对应方法中设置断点、观察向前看符号，一切尽在掌控。两种范式的优劣只有亲身体验才能深刻理解。

Oberon-0 语言设计的简洁与严谨也给了我很大启发。Wirth 通过 IF-END 显式闭合结构从根本上消除了悬挂 else 问题，通过四层表达式文法将运算符优先级自然编码在语法中，通过 {} 和 [] 扩展使文法天然适用于递归下降分析——这些设计决策使得 Oberon-0 成为教学和实验的理想语言，也让我对”好的语言设计”

有了更具体的认知。

最后，软件工程方面的收获同样宝贵。14 个异常类的三层继承体系、Scanner-Adapter 适配器模式、两阶段构建策略、javadoc 文档生成——这些实践让我认识到，编译技术不仅是一门理论深厚的学科，也是一个对软件工程能力要求极高的系统工程领域。

参考文献

[1] Wen jun Li. Lab05-rose: Oberon-0 逆向工程工具 rose, 2022. Version 3.01.